Motor de Captura de Logs de Operações

Arquitetura de um motor autônomo, configurável e de alta eficiência para sistemas web

Documento técnico · novembro de 2026

Sim, é possível construir um motor assim. É o modelo que ferramentas de auditoria e observabilidade maduras usam. Três princípios definem o projeto:

1. Desacoplado

O sistema principal não depende do motor. Se o motor cair, o sistema continua funcionando (*fail-open*).

2. Assíncrono

A captura nunca grava no banco dentro da requisição do usuário. Ela só enfileira em memória, o que leva microssegundos.

3. Guiado por configuração

Regras declarativas definem o que capturar, o que mascarar e por quanto tempo guardar, sem deploy.

1. Arquitetura

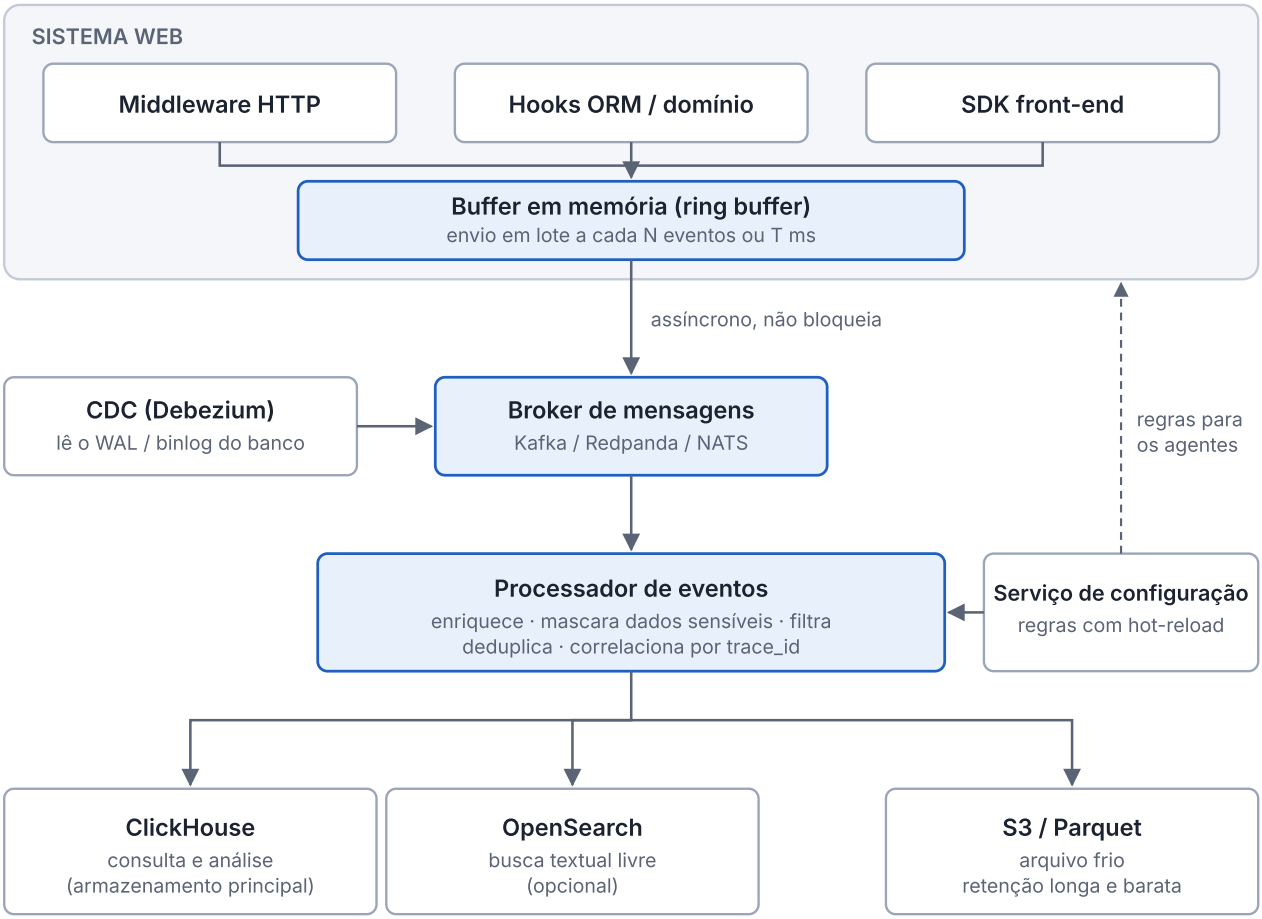


Figura 1. Fluxo dos eventos, da captura ao armazenamento.

2. Como ele captura os eventos

Há quatro pontos de captura. Eles se complementam.

Ponto	O que captura	Intrusão no código	Contexto de usuário
Middleware HTTP / API Gateway	Rota, método, status, latência, IP, usuário, payload (filtrado)	Mínima, uma linha de registro	Completo
Hooks do ORM / eventos de domínio	"Usuário X alterou o campo Y do pedido Z de A para B"	Baixa a média	Completo
CDC (Change Data Capture)	Toda alteração no banco, com estado antes e depois	**Zero**, lê o log de transações	Precisa correlacionar
SDK no front-end	Cliques, navegação, erros de JavaScript	Baixa	Completo

CDC: o ponto mais autônomo. O Debezium lê o WAL do PostgreSQL ou o binlog do MySQL sem tocar no código e sem gerar carga de consultas. A limitação é que o banco não sabe *quem* fez a alteração. Para resolver, a aplicação marca a transação com o usuário (por exemplo, `SET LOCAL app.user_id = '123'`) ou grava o `trace_id`. O processador depois junta o evento do CDC com o evento HTTP pelo `trace_id` ou pelo `txid`.

O segredo da eficiência: o caminho rápido

Exemplo em Go. A chamada dentro da requisição só coloca o evento num canal com capacidade fixa. Uma goroutine separada agrupa e envia os lotes.

```
type Capturer struct {
    ch chan Event // buffer com capacidade fixa, ex.: 65.536
}

// Chamado dentro da requisição: custo de nanossegundos, nunca bloqueia
func (c *Capturer) Emit(e Event) {
    select {
    case c.ch ← e:
    default:
        droppedCounter.Inc() // buffer cheio: descarta ou grava em spool no disco
    }
}

// Goroutine separada: agrupa em lotes e envia
func (c *Capturer) run() {
    batch := make([]Event, 0, 1000)
    tick := time.NewTicker(200 * time.Millisecond)
    for {
        select {
        case e := ←c.ch:
            if batch = append(batch, e); len(batch) == cap(batch) {
                c.flush(batch); batch = batch[:0]
            }
        case ←tick.C:
            if len(batch) > 0 { c.flush(batch); batch = batch[:0] }
        }
    }
}
```

Técnicas que garantem a eficiência:

- **Envio em lote:** mil eventos por envio, em vez de mil envios.
- **Serialização compacta:** Protobuf ou MessagePack em vez de JSON, com compressão zstd ou lz4.
- **Spool local em disco:** se o broker ficar fora do ar, os eventos não se perdem.
- **Amostragem configurável** para eventos de alto volume e baixo valor, como leituras e health checks.
- **Backpressure com métricas:** eventos descartados viram um alerta, nunca uma perda silenciosa.

3. Configuração

As regras ficam num arquivo declarativo. O serviço de configuração publica as mudanças e os agentes recarregam as regras sem reiniciar.

```yaml
capture:
  http:
    include: ["/api/**"]
    exclude: ["/api/health", "/api/metrics"]
    methods: [POST, PUT, PATCH, DELETE]   # leituras ficam de fora
    sample_rate: { GET: 0.05 }            # se incluir GET, guarda 5%
    body: { max_bytes: 4096 }
  entities:
    Pedido:   { track: all, diff: true }
    Usuario:  { track: [email, perfil, ativo], diff: true }
    Produto:  { track: [preco], diff: true }
masking:
  fields: [senha, cpf, cartao, token]
  strategy: hash   # hash | redact | partial (***.***.123-**)
retention:
  audit: 5y
  operacional: 90d
```

Formato do evento

```json
{
  "event_id":   "0192f3a1-...",            // UUIDv7, ordenável por tempo
  "ts":         "2026-11-05T14:32:10.123Z",
  "trace_id":   "4bf92f35...",             // OpenTelemetry: liga HTTP, ORM e CDC
  "tenant":     "empresa-x",
  "actor":      { "id": "u123", "tipo": "usuario", "ip": "10.0.0.5" },
  "action":     "pedido.atualizar",
  "resource":   { "tipo": "Pedido", "id": "98765" },
  "changes":    { "status": ["ABERTO", "FATURADO"] },
  "result":     "sucesso",
  "latency_ms": 42
}
```

4. Qual banco usar? O mesmo do sistema?

Em regra, não. Os motivos:

Problema	Por quê
Concorrência de carga	Log é escrita massiva e contínua. No mesmo banco, disputa I/O, locks e cache com as transações do negócio.
Padrão de acesso diferente	O banco transacional é otimizado para leituras e escritas pontuais. Log é *append-only* com consultas agregadas por período, e bancos colunares fazem isso de 10 a 100 vezes melhor.
Volume	Logs crescem muito mais que os dados do negócio e inflam backup, vacuum e réplicas.
Segurança e integridade	Quem administra o banco do sistema conseguiria apagar a própria trilha de auditoria.
Acoplamento	Se o banco principal cai, os logs que explicariam a queda caem junto.

Recomendação

- **ClickHouse** é a primeira escolha. Ele é colunar, comprime de 10 a 20 vezes, ingere milhões de linhas por segundo e tem `TTL` nativo para retenção.

```
CREATE TABLE eventos (
    ts            DateTime64(3),
    tenant        LowCardinality(String),
    actor_id      String,
    action        LowCardinality(String),
    resource_type LowCardinality(String),
    resource_id   String,
    trace_id      String,
    changes       String CODEC(ZSTD(3)),
    result        LowCardinality(String)
) ENGINE = MergeTree
PARTITION BY toYYYYMM(ts)
ORDER BY (tenant, resource_type, resource_id, ts)
TTL ts + INTERVAL 5 YEAR;
```

- **OpenSearch ou Elasticsearch**, se o requisito principal for busca textual livre nos logs.
- **TimescaleDB**, se a equipe só domina PostgreSQL e o volume é moderado.
- **S3 com Parquet** para o arquivo frio barato, consultável por ClickHouse, DuckDB ou Athena.

Quando faz sentido usar o mesmo banco

1. **Sistema pequeno, com poucos milhares de eventos por dia.** Use um *schema* separado e tabelas particionadas por mês, com gravação ainda assíncrona. Montar Kafka e ClickHouse aqui seria exagero.
2. **Auditoria com garantia legal, em que nenhum evento pode ser perdido.** Use o padrão **Transactional Outbox**: o evento é gravado numa tabela `outbox` na *mesma transação* da operação de negócio. Se a operação foi confirmada, o log existe. Um processo separado lê a outbox, envia ao broker e limpa a tabela. O banco principal funciona só como passagem temporária, e o armazenamento definitivo fica em outro lugar.

5. Pontos que costumam ser esquecidos

- **Separe auditoria de telemetria.** Auditoria ("quem fez o quê") não pode ser amostrada nem perdida, e vale considerar outbox para ela. Telemetria operacional pode ser amostrada e descartada sob carga.
- **Imutabilidade.** Use encadeamento de hashes, em que cada evento guarda o hash do anterior e qualquer adulteração fica detectável. Combine isso com permissões só de inserção e, se preciso, armazenamento WORM (S3 Object Lock).
- **LGPD.** Mascare os campos sensíveis na origem, antes de os dados saírem da aplicação, e defina retenção por tipo de evento.
- **Monitore o próprio motor.** Acompanhe eventos por segundo, eventos descartados, atraso do consumidor no Kafka e tamanho do spool.
- **Use OpenTelemetry** como padrão de contexto. O `trace_id` é o que permite ligar o clique no front-end, a chamada à API e a alteração no banco.

Resumo

Decisão	Recomendação
Captura	Middleware HTTP + hooks do ORM, complementados por CDC (Debezium) para cobertura total do banco
Caminho da requisição	Só enfileira em memória; envio assíncrono em lote; fail-open
Transporte	Kafka, Redpanda ou NATS JetStream, com spool local em disco
Configuração	Regras declarativas (YAML/JSON) com hot-reload
Armazenamento	ClickHouse (quente), S3/Parquet (frio); banco separado do sistema
Garantia de auditoria	Transactional Outbox + encadeamento de hashes